

```

#include <iostream>
#include <fstream>
#include <string>

using namespace std;

// -----
// Email: one message in the CEO's inbox plus its priority info.
// -----
class Email {
private:
    string sender;      // sender category, e.g. "Boss"
    string subject;     // subject line (may contain spaces)
    string date;        // date as given, MM-DD-YYYY
    int rank;           // numeric priority from sender category
    long dateValue;     // date encoded as YYYYMMDD for comparison

    // Map sender category to a rank (higher = read first).
    static int rankFor(const string &s) {
        if (s == "Boss") return 5;
        if (s == "Subordinate") return 4;
        if (s == "Peer") return 3;
        if (s == "ImportantPerson") return 2;
        if (s == "OtherPerson") return 1;
        return 0; // unknown category: lowest
    }

    // Convert MM-DD-YYYY to YYYYMMDD so dates compare as integers.
    static long valueFor(const string &d) {
        if (d.size() < 10) return 0; // malformed: treat as oldest
        int month = stoi(d.substr(0, 2));
        int day = stoi(d.substr(3, 2));
        int year = stoi(d.substr(6, 4));
        return year * 10000L + month * 100L + day;
    }

public:
    Email() : rank(0), dateValue(0) {} // default needed for array
    allocation

    Email(const string &s, const string &subj, const string &d)
        : sender(s), subject(subj), date(d),
          rank(rankFor(s)), dateValue(valueFor(d)) {}

    // True if this email should be read before 'other'.
    bool higherPriorityThan(const Email &other) const {
        if (rank != other.rank) return rank > other.rank; // category
first
        return dateValue > other.dateValue;                // then newest
first
    }

    void display() const {
        cout << "Next email:\n"

```

```

        << "Sender: " << sender << "\n"
        << "Subject: " << subject << "\n"
        << "Date: " << date << "\n";
    }
};

// -----
// MaxHeap: list-based (dynamic array) binary max heap of Emails.
// -----
class MaxHeap {
private:
    Email *items;    // underlying array
    int count;       // elements stored
    int capacity;    // allocated slots

    // Double the array when full.
    void grow() {
        int newCap = capacity * 2;
        Email *bigger = new Email[newCap];
        for (int i = 0; i < count; i++) bigger[i] = items[i];
        delete[] items;
        items = bigger;
        capacity = newCap;
    }

    void swapItems(int i, int j) {
        Email tmp = items[i];
        items[i] = items[j];
        items[j] = tmp;
    }

    // Move element at i up until its parent outranks it.
    void siftUp(int i) {
        while (i > 0) {
            int parent = (i - 1) / 2;
            if (!items[i].higherPriorityThan(items[parent])) break;
            swapItems(i, parent);
            i = parent;
        }
    }

    // Move element at i down until both children rank lower.
    void siftDown(int i) {
        while (true) {
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            int best = i;
            if (left < count &&
items[left].higherPriorityThan(items[best])) best = left;
            if (right < count &&
items[right].higherPriorityThan(items[best])) best = right;
            if (best == i) break;
            swapItems(i, best);
            i = best;
        }
    }
};

```



```

public:
    void process(string line) {
        // Strip trailing CR/LF so Windows line endings don't break
        matching.
        while (!line.empty() && (line.back() == '\r' || line.back() ==
'\n'))
            line.pop_back();
        if (line.empty()) return;

        if (line.compare(0, 6, "EMAIL ") == 0) {
            addEmail(line.substr(6));
        } else if (line == "NEXT") {
            if (heap.empty()) cout << "No emails to read.\n";
            else heap.peek().display();
        } else if (line == "READ") {
            if (!heap.extractMax()) cout << "No emails to read.\n";
        } else if (line == "COUNT") {
            cout << "There are " << heap.size() << " emails to read.\n";
        } else {
            cerr << "Warning: unrecognized command ignored: " << line <<
"\n";
        }
    }
};

int main(int argc, char *argv[]) {
    istream *in = &cin;
    ifstream file;
    if (argc > 1) {
        file.open(argv[1]);
        if (!file) {
            cerr << "Error: could not open file " << argv[1] << "\n";
            return 1;
        }
        in = &file;
    }

    CEOInbox inbox;
    string line;
    while (getline(*in, line)) inbox.process(line);
    return 0;
}

```